

Chapter 7

VARIOUS TYPES OF VULNERABILITIES

Application Vulnerabilities

1. Memory injection:

- These attacks involve the secret insertion of malicious code into a program's memory space, allowing attackers to exploit vulnerabilities to gain unauthorized access or execute arbitrary commands.
- This unauthorized access and manipulation can lead to
 - a. data breaches,
 - b. loss of data integrity, and
 - c. provide a gateway for broader system compromises by malicious actors.

Example of memory injection attack: Code Red worm that affected Microsoft IIS web server in 2001

Application Vulnerabilities

2. Buffer overflow:

- A **buffer overflow** occurs when a program writes more data into a buffer than it can hold, causing excess data to overwrite adjacent memory. This can lead to crashes, system instability, or allow attackers to execute arbitrary code.
- A notable example is the **SQL Slammer worm**, which exploited a buffer overflow in **Microsoft SQL Server**, causing widespread internet disruption.
- **Prevention:** input validation, proper memory management, and using secure programming practices or languages with built-in safeguards.

Application Vulnerabilities

3. Race conditions:

- A **race condition** occurs when multiple threads or processes access shared data at the same time, causing unpredictable results.
- A scenario where one person is viewing a file's attributes, while, simultaneously, another person accesses the same file. This phenomenon is referred to as Time-of-check (TOC) or time-of-use (TOU).
- **Example:** Airline reservation systems may allow double booking of the same seat.
- **Prevention:** synchronization techniques like locks and proper handling of shared resources.

Application Vulnerabilities

4. Malicious update:

- It occurs when a seemingly legitimate software update contains hidden code or alterations crafted by malicious actors.
- Once installed, it can give attackers unauthorized access, steal data, or create backdoors.
- A notable example is the **CCleaner supply chain attack**, where attackers compromised the update mechanism of **CCleaner**.
- **Prevention:** verify updates from trusted sources, use digital signatures, and enable strong security measures like multi-factor authentication.

Web-Based Vulnerabilities

- It serve as gateways for digital intruders.
- The two prominent threats are **Structured Query Language Injection (SQLI)** and **Cross-Site Scripting (XSS)**.
- **SQLI** is a type of cyberattack that occurs when an attacker exploits vulnerabilities in a website or an application's input fields to manipulate the SQL queries executed on the backend database.
- SQLI works as follows:
 - 1. Input fields:** Many web applications take user input through forms or URL parameters. These inputs are often used to construct SQL queries to interact with a database.
 - 2. Malicious input:** An attacker enters specially crafted input (often containing SQL code) into these input fields
 - 3. Query manipulation:** If an application doesn't properly validate or sanitize input, the malicious SQL code becomes a part of the query executed on a database
 - 4. Data exposure:** Depending on the type of attack, an attacker can extract sensitive information, modify or delete data, or even gain administrative control over a database

Web-Based Vulnerabilities

- Example:
- `user_input = request.GET['product_name']`
- `query = "SELECT * FROM products WHERE name = '" + user_input + '";" result = execute_query(query)`
- Once the attacker launches the code above, they notice that the website is vulnerable to SQL injection and decides to exploit it.
- Instead of searching a legitimate product name, the attacker enters the following malicious input:
- `SELECT * FROM products WHERE name = " OR '1=1'; --";`
- This bypasses filters and returns all data.

Web-Based Vulnerabilities

SQLI attacks can be mitigated in the following ways:

1. Stored procedure:

- A stored procedure is a database object that encapsulates a sequence of SQL statements.
- These statements can perform a variety of operations, including data manipulation, queries, and transactions.
- Stored procedures are stored in the database and can be invoked from applications or other database objects.

2. Input validation:

- Validate and sanitize all user inputs before using them in SQL queries.
- Use parameterized queries or prepared statements, which automatically handle input sanitization.

Web-Based Vulnerabilities

XSS:

- XSS represents a significant security concern in web applications due to its potential for malicious code injection, executed in the context of a victim's browser.
- This can lead to the compromise of user data, session hijacking, and even the defacement of websites.
- XSS can use the `<script>` and `</script>` HTML tags and can include JavaScript in between, which can be identified with a `.js` extension.
- For an example of XSS, imagine a web application that allows users to post comments on a forum. The comments are displayed to other users on the website. However, the website doesn't properly sanitize or encode user input, creating an XSS vulnerability.
- A malicious user posts the following comment:

```
html

<script>

    alert('XSS Attack!');

</script>
```

Virtualization Vulnerabilities

- Virtualization allows multiple virtual machines (VMs) to run on a single physical server, optimizing resource utilization, enhancing flexibility, and streamlining IT operations.
- However, like any technology, virtualization also introduces vulnerabilities that require careful consideration to ensure the security and stability of systems.

Vulnerabilities associated with virtualization and the strategies to mitigate these vulnerabilities

1. VM escape:

- A vulnerability where an attacker breaks out of a virtual machine and gains access to the host system or other VMs.
- Attackers may move between VMs or from a VM to the host system (east-west movement).
- Can lead to unauthorized access, data breaches, and full system compromise affecting host and multiple VMs.

2. Resource reuse:

- Sharing resources in virtualization can cause performance and security issues if not properly managed.
- Reusing storage without clearing data may expose sensitive information to new VMs.
- One VM consuming excessive resources can reduce performance of other VMs on the same host.
- Overuse of shared resources can lead to system slowdown or failure of other VMs

3. Vm Sprawl:

- Uncontrolled growth of virtual machines in a virtualized environment.
- Leads to management difficulties, higher resource usage, and security risks.
- Excessive or unregulated creation of VMs.
- Requires automated provisioning, resource monitoring, regular audits, and strong governance to control VM usage and ensure efficiency.

Note: Implement strong access controls, network segmentation, and VM isolation to mitigate the risk

Cloud-Specific Vulnerabilities

- As businesses harness the power of cloud computing, there is a risk of criminals exploiting the cloud for financial gain.

1. **Shared tenancy risk:**

- Multiple customers share the same physical cloud infrastructure in isolated virtual environments.
- **Security issue:** Poor data protection can lead to side-channel attacks between tenants.
- **Impact:** A breach in one tenant's environment may unintentionally affect others.
- **Requirement:** Strong isolation mechanisms and security controls are needed to ensure data confidentiality and integrity.

2. **Inadequate configuration management:**

- Cloud services have complex settings and permissions.
- **Risk:** Misconfiguration or lack of understanding can expose resources or open ports.
- **Impact:** Creates vulnerabilities that allow unauthorized access or malicious attacks.

3. **Identity and access management (IAM) flaws:**

- Weak or misconfigured access controls in cloud systems.
- **Causes:** Misconfigured permissions, compromised credentials, and weak authentication.
- **Impact:** Enables unauthorized access and account compromise.

4. **Cloud Access Security Broker (CASB):**

- A tool that enforces an organization's security policies in cloud environments.
- Bridges on-premises systems and cloud services.
- Provides centralized security control where cloud lacks unified governance.
- Monitors cloud users, ensures security compliance, and checks that devices are patched.
- Offers visibility and control across all cloud platforms.

Supply Chain Vulnerabilities

- **Supply chain is a** network of organizations, people, activities, and resources involved in producing and delivering goods or services.
- Businesses rely on multiple providers such as service, hardware, and software vendors.
- Improves efficiency and innovation through interconnected systems.
- Introduces vulnerabilities that can have wide-reaching security impacts.
 - **Service provider vulnerabilities:** Reliance on third-party services can lead to weak security controls, causing data breaches, service disruptions, and unauthorized access.
 - **Hardware provider vulnerabilities:** Compromised or counterfeit hardware can introduce security flaws, affecting system integrity, resilience, and data confidentiality.
 - **Software provider vulnerabilities:** Software vendors must provide a software bill of materials (SBOM) to identify components used and improve transparency and security.

Mobile Device Vulnerabilities

- Mobile devices have seamlessly integrated into our modern lives, serving as essential conduits to communication, information, and entertainment.
- However, this convenience comes hand in hand with vulnerabilities that can compromise our digital security.

Significant threats to mobile device security

- **Jailbreaking:**
 - **Jailbreaking** applies specifically to Apple devices and allows users to bypass manufacturer or operating system restrictions, providing more control over the device. This is commonly known as **unlocking a device**.
 - This freedom exposes the device to significant security risks.
- **Rooting:**
 - Rooting allows users to bypass manufacturer or operating system restrictions on Android devices, providing more control over a device. This is commonly known as **unlocking a device**.
 - This freedom exposes the device to significant security risks.
- **Sideload:**
 - Sideload is generally associated with Android devices utilizing Android Application Package (APK) files.
 - While applications can also be sideloaded on Apple devices, the practice directly violates Apple's terms and conditions and voids the device's warranty. Voiding a device's warranty eliminates official support from the manufacturer, meaning the device may no longer receive security patches, bug fixes, or updates, leaving it vulnerable to new threats and exploits.

A device that has been subject to either jailbreaking or rooting may be susceptible to the following vulnerabilities:

- **Counterfeit apps:** Unregulated stores can host third-party counterfeit or modified versions of legitimate apps that carry hidden malware
- **Software instability:** Unauthorized software modifications can destabilize a device's operating system, leading to crashes and erratic behavior
- **Security compromises:** Jailbreaking undermines device security measures, making it easier for malware and viruses to exploit vulnerabilities
- **Stolen data:** Purchasing apps from unregulated stores may expose users to data breaches and financial theft

- Sideloaded, jailbreaking, and purchasing from unregulated stores may offer short-term benefits, but the long term security implications cannot be ignored.
- **Mobile device management (MDM)** solutions set policies for the installation and protection of mobile devices, such as password policies on password length or remote wipes for lost or stolen devices to revert them to the factory setup.
- They can also roll out updates for the devices that they manage. For example, they can prevent a camera from being used on mobile devices or a smartphone from being able to send/receive texts.
- The cloud version of MDM is the CASB. By being vigilant, making informed choices, and embracing security-conscious practices, we can navigate the mobile landscape with confidence and safeguard our digital experiences.